Where Does the Time Go? A Pre-Registered Study of Per-Step Cost in LLM Inference Serving

Patrick Niyongabo
Independent researcher, Portland, Oregon
https://github.com/pniyongabo/BottleneckShift

October 2026

Abstract

When an LLM inference server slows down under load, it is often unclear where the time goes. We measured vLLM serving a 0.5B-parameter model on one GPU in a series of pre-registered experiments totaling 18,900 measured requests. Most runs were observed from the client, the server's phase metrics, and GPU and CPU telemetry, and the last experiments added a per-step scheduler log. Every prediction was published before its data and scored after.

First, from 8 to 32 concurrent requests, past throughput saturation, the added latency moves into each engine step: the share of latency before the first token falls from 0.44 to 0.24 while median inter-token latency rises from 4.9 to 55 ms. Second, the scheduler's token budget decides where that delay appears. At 32 concurrent requests, a larger budget gives a 7 ms median and 18% more throughput but a 169 ms 99th percentile, and with the smallest budget queueing takes two thirds of each request's latency. Third, with one request at a time, a fixed-size prompt chunk costs up to 1.6× more at deeper context, in line with a count of attention FLOPs, and a cost model fit on those data predicts the 99th-percentile inter-token latency at 8 concurrent requests within 5% (post hoc). The pre-registered test that a step's cost tracks its scheduled tokens failed at 8 concurrent requests and passed at 32 with a timing measure we later found lags by one step.

Of 59 scored predictions, 40 held. We report the failures in full, including that measurement flaw. All GPU time for the study so far cost $2.04.

1 Introduction

LLM serving systems batch many requests through one model. Each engine step generates one token for every decoding request and may also process part of a new prompt. Service targets are written in time to first token (TTFT) and time per output token (TPOT), so an operator who sees latency rise needs to know which phase grew. Past saturation the excess work is the same either way; the scheduler decides whether it shows up as waiting before the first token or as slower steps.

We ask when the dominant source of client-visible delay changes, and which measurements can tell the causes apart, for one setup: vLLM [5] serving Qwen2.5-0.5B-Instruct on one RTX A4000. Each experiment's predictions and thresholds were published before its data, each run was measured from three views, and every prediction was scored afterward. Our contributions are:

- a pre-registered, three-view measurement method with a per-step scheduler log (Section 3);
- where latency goes under load from C8 to C32, and why that shift is bounded (Section 4);
- how the token budget trades median against tail latency (Section 5);
- what sets a step's cost: scheduled tokens and context, a FLOP count, and a post hoc step-cost model (Section 6);
- a ledger of all 59 scored predictions and what the misses taught us (Section 8).

The code, configurations, protocols, and reports are public.[1]

[1] https://github.com/pniyongabo/BottleneckShift

2 Background

Phases and steps. vLLM runs in *engine steps*: each step, the scheduler picks the tokens to process and the model runs one forward pass over them. A request waits in a *queue*; its *prefill* then processes the prompt and produces the first output token, and its *decode* produces each remaining token, one per step. With chunked prefill [3], a long prompt is split across steps, so one step can carry both decode tokens and prompt chunks. vLLM 0.29.0 enables asynchronous scheduling by default in our configuration: it schedules step $n{+}1$ before it processes step n's results.

Token budget. The *token budget* B (`max_num_batched_tokens`) is the most tokens one step may process, counting one per decoding request and every prompt token in the step's chunks. vLLM's default for its API server on GPUs under 70 GB, as here, is 2048. The scheduler fills each step with running requests first and then with waiting prompts, until the budget is used. A step that carries a large chunk takes about as long as a prefill, and every request decoding in it waits. vLLM's documentation says smaller budgets give better ITL and larger ones better TTFT and throughput [10].

Metrics. We use the definitions of vLLM's benchmark. TTFT runs from sending a request to its first streamed output, and E2E to its last. Inter-token latency (ITL) is each gap between consecutive streamed outputs of a request, usually one token each; time per output token (TPOT) is a request's average gap, $(\text{E2E} - \text{TTFT})/(n-1)$ for n output tokens. We report ITL p50 and p99, because averages hide jitter [1]. The benchmark pools the gaps of all requests in a run, and one step produces a gap for every request decoding in it, so p99 reflects a run's slowest few steps; a mean ITL nearly equals the mean TPOT. The *share of latency before the first token* is $\sum \text{TTFT} / \sum \text{E2E}$ over a run. vLLM's server histograms split each request into queue, prefill (first scheduled to first token), and decode (first token to last) time.

3 Method

Hardware and software. Each session rented a virtual machine with one NVIDIA RTX A4000 (Ampere, 16 GB, 140 W board power), 4 vCPUs, and 20 GiB of RAM (recorded for Milestones 1–3), running Ubuntu 24.04 (Linux 6.14). Sessions ran on several such machines, each with its own physical GPU; Phase C reused Phase B's. The software was vLLM 0.29.0 with PyTorch 2.13.0, CUDA 13.0, and Python 3.12.3, and NVIDIA driver 580.178.04 for Milestones 1–3 and 580.196.07 for Milestone 5; `nvidia-smi` is the version bundled with each driver. The model was Qwen2.5-0.5B-Instruct in bf16.

Workload. The client was `vllm bench serve` on the same host, issuing a closed-loop workload: up to a fixed number of requests (the concurrency, C1 to C32) are kept in flight, and a new one is sent as soon as one finishes. Each run sends 100 requests (50 for the decode-heavy workload), so a run at C32 starts with a burst of 32 requests and ends as the last ones drain. Prompts are synthetic, with exact input and output lengths and a fixed seed.

Controls. Milestone 1 compared C1 with C8 in forward and reverse order, with three repetitions and 15-second cooldowns, and found 6.65× higher output throughput at C8. It ran with prefix caching on, an unpinned model revision, and the model's sampling defaults (temperature 0.7, top-p 0.8, top-k 20, repetition penalty 1.1), any of which could bias the result. Milestone 2 repeated it with prefix caching off, the server's model and tokenizer revisions pinned, greedy sampling with the model's generation defaults ignored (`-generation-config vllm`), and warm-ups at each concurrency. The effect held (about 6.5×), and these controls apply to every later run.

Pre-registration. Each phase's protocol was merged into the public repository before its data were collected, and the merge time is the evidence. Milestones 1 and 2 stated expected signatures. From Milestone 3 on, protocols state predictions with thresholds and falsification rules, and each report scores every prediction as confirmed, mixed, or not confirmed (a few are qualified, such as "not testable") and labels interpretations

made after seeing the data as post hoc. A final reanalysis of the retained data was not pre-registered, and its results are marked post hoc.

Measurement. From Milestone 3 on, each run is measured three ways: by the client (TTFT, ITL, E2E, and output throughput); by vLLM's Prometheus histograms of queue, prefill, and decode time per request, scraped before and after each run; and by `nvidia-smi` GPU utilization every 200 ms and CPU utilization every second. vLLM's per-iteration token histogram credits a request's whole prompt to the step that emits its first token, so it cannot show how many prompt tokens a step processed. For Milestone 5 we therefore added an opt-in vLLM plugin that logs each engine step's scheduled tokens, split into prefill and decode, and its requests. Part 2 added each prefill chunk's context position and the time the step's results returned. From these we compute a step's *execution time*: from when it could start (it was scheduled and the previous step's results had arrived) until its own results arrived.

Validation. Before analysis, a validator checks each series for zero failures and exact token lengths, server request counts that match the prompts sent, warm-ups, telemetry coverage (gaps of up to 1 s for GPU and 3 s for CPU samples), and, for Milestone 5, step-log coverage of every run. Raw results are archived with a SHA-256 checksum, verified after transfer, before the machine is deleted. The study comprises 18,900 measured requests.

4 Where latency goes under load

Decode dominates each request, even with long prompts. We first compared a prefill-heavy workload (2048 input and 32 output tokens) with a decode-heavy one (64 and 512) at C1 and C8. We had predicted that time before the first token would dominate the prefill-heavy workload, but a request running alone spent about 40 ms in prefill and about 122 ms in its 31 decode steps. A cost estimate predicts this [8]. Prefill of 2048 tokens needs about 1.6 TFLOP (two FLOPs per weight per token, plus attention), or 21 ms at the A4000's dense 16-bit tensor peak of 77 TFLOPS. Each decode step must read about 1 GB of weights, at least 2.2 ms at 448 GB/s, or 68 ms for 31 steps [6]. Even at these ideal rates prefill is about a quarter of the request, so the prediction could have been made in advance.

From C1 to C8 the share of latency before the first token rose from 31% to 45%. (At C1 the client's 54 ms TTFT includes about 14 ms of request handling outside the server's prefill time.) The decode-heavy workload gained 6.46× throughput for 10% higher TPOT; the prefill-heavy one gained only 2.8×, with queueing and a tenfold rise in ITL p99. At C8 much of a request's decode time is spent in steps that also carry other requests' prompt chunks, so a per-request phase breakdown books GPU prefill work as decode time. In Milestone 5's step log, 23% of C8 steps at this prompt length carried prompt work.

Past saturation, the added latency moves into each step. Raising concurrency to C16 and C32, we predicted that time before the first token would come to dominate. Instead, its share fell from 0.44 at C8 to 0.35 at C16 and 0.24 at C32. Output throughput peaked at C16 (583 tokens/s) and fell slightly at C32 (555), and median ITL rose from 4.9 ms at C8 and 5.5 ms at C16 to 55 ms at C32 (Figure 1). A pre-registered sweep (C8, C12, C16, C24, C32) on a different machine reproduced these values within about 1% and placed the jump between C16 (5.5 ms) and C24 (45.8 ms). The jump is abrupt because the gaps form two clusters, below 10 ms from decode-only steps and 40–80 ms from prompt-carrying steps. The share of gaps above 20 ms rose from 31% at C16 to 56% at C24, while mean ITL rose steadily: 8.7, 13.3, 18.0, 30.5, and 45.1 ms from C8 to C32 (post hoc).

The shift is bounded. At B=2048 a step costs at most about 56 ms (ITL p99 was 58 ms at C32), so a request's 31 decode steps take at most about 1.8 s. Past saturation throughput no longer grows, so by Little's law each added request in flight adds latency, and once steps are full it must go into the wait for the first token. Over full runs, mean TTFT doubled from C8 (215 ms) to C32 (431 ms), but most of that comes from the burst of requests that starts each run. Without each run's first and last C requests, it rose only from 212 to 255 ms, and ITL changed by about 1% (post hoc). The per-step shift therefore describes C16

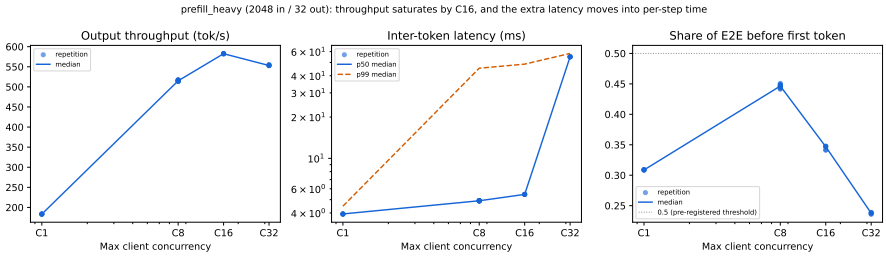


Figure 1: Prefill-heavy workload (2048/32 tokens) against concurrency: output throughput, ITL p50 and p99, and the share of latency before the first token. The C1 points and six of the twelve C8 points come from the contention study's control series.

Table 1: Token budget at C32, prefill-heavy workload (2048/32 tokens). Medians of three repetitions, which agree within about 1% (mean TTFT within 4%).

Budget	ITL p50 (ms)	ITL p99 (ms)	Mean TTFT (ms)	Output tok/s
512	15.8	17.8	1378.8	496.2
2048	55.4	58.6	432.7	550.8
8192	6.9	168.7	686.1	652.5

to C32 at this budget. Beyond C32, and under open-loop load, we expect delay to return to TTFT; we did not test this.

5 The token budget trades median against tail

If the jump in per-step time comes from prompt-carrying steps, the token budget (Section 2) should control it. Its default, 2048, equals the prompt length above.

At C32 we ran budgets of 512, 2048, and 8192 (Table 1). A 512-token budget spreads prompt work over many small steps: ITL is nearly flat (p50 15.8 ms, p99 17.8 ms), but TTFT triples, throughput falls 10%, and queueing takes two thirds of each request's latency (1.2 of 1.8 s). At 2048, most steps carry a full 2048-token chunk (in Milestone 5's step log, 77% of steps carried prompt work), so p50 and p99 are both near 55–59 ms. At 8192, several prompts share fewer, larger steps: the median falls to 6.9 ms and throughput rises 18%, but p99 rises to 169 ms. The budget with the best median has the worst tail, so the documentation's ITL guidance holds for p99 only. Against its TTFT guidance, 8192 also raised mean TTFT relative to 2048; without each run's start-up burst, mean TTFT was 1,542, 217, and 609 ms for 512, 2048, and 8192 (post hoc). By mean TPOT, the order is 512 (15 ms), 8192 (26 ms), and 2048 (45 ms).

We had predicted that ITL p50 and p99 would both order 512 $<$ 2048 $<$ 8192. The p99 ordering held and the p50 ordering did not. The manipulation check relied on the iteration-token histogram and failed as specified, so under the pre-registered rule the test of the mechanism was inconclusive. If we accept server-log evidence that the budgets were applied, a deviation from the protocol, the pre-registered form of the step-size explanation (a p50 that rises with the budget) is unsupported. The open mechanism motivated Milestone 5.

Prompt length against budget. Milestone 5 varied the prompt length $L \in \{1024, 2048, 4096\}$ at budgets 2048 and 8192 with the step log on (Table 2). At C32 and B=2048, ITL p99 was 58 ms for 2048-token prompts and 65 ms for 4096-token prompts: past the budget, prompts are cut into budget-sized chunks, so the tail grew only 11%. At B=8192 the tail was about 3× higher (170 and 200 ms), and the median stayed below 10 ms. We predicted that 1024-token prompts would have a tail at least 25% lower than 2048-token prompts. It was 19% lower, because the scheduler packed two 1024-token prompts into one 2048-token step (post hoc; Section 6). At C8 and B=2048, ITL p50 was 4.6–5.7 ms for every prompt length, as predicted, against 55–57 ms at C32 for $L \geq 2048$. The tail fell less, to 40–54 ms, and packing again limited the L=1024 tail (11% lower, against a predicted 25%). At C8 with B=8192, prompt length did move the tail: p99 rose from 60 to 178 ms between L=1024 and L=4096.

Table 2: Prompt length L against budget B at C32 and C8 (32 output tokens). ITL p50 / p99 in ms; medians of three repetitions.

B	L	C32	C8
2048	1024	6.1 / 47.4	4.6 / 40.4
2048	2048	55.3 / 58.3	4.9 / 45.5
2048	4096	57.4 / 64.7	5.7 / 53.8
8192	1024	5.7 / 151.3	4.6 / 60.4
8192	2048	7.0 / 169.5	4.9 / 120.3
8192	4096	9.7 / 199.9	5.5 / 177.5

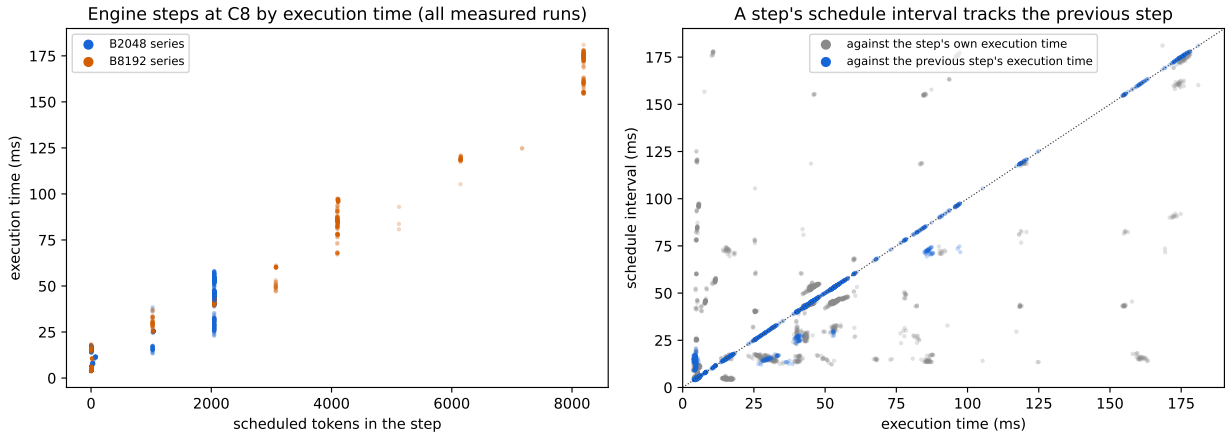


Figure 2: Engine steps at C8 (post hoc). Left: each step's execution time against its scheduled tokens. Right: each step's schedule interval against its own execution time (gray) and the previous step's (blue); the dotted line is equality.

6 What sets the cost of a step

Step size. The step log confirmed that the budget was applied exactly: no step scheduled more than B tokens, and each run's logged prompt tokens equalled the number sent. Milestone 5's primary prediction was that a step's cost tracks its scheduled tokens, measured by the interval from one scheduling call to the next. Pooled over the seven C32 series, steps of at least 1024 tokens were followed by a median interval of 56 ms, against 5.7 ms for steps of at most 64 tokens (9.8×, Spearman 0.87), and the prediction passed. At C8 the ratio was 9.0× but the Spearman correlation was 0.57, below its threshold of 0.7, which under the protocol's rule rejects the hypothesis's first clause at C8.

A lagged measure. Under asynchronous scheduling (Section 2), the interval credited to step n partly reflects the cost of step $n-1$. In the C8 block the engine scheduled 97–100% of steps this way, and at B=2048 a step's interval correlated 0.96–0.99 with the previous step's execution time, against 0.69–0.82 with its own (post hoc; Figure 2). Some intervals are too short to be a step's own cost: at C8 with B=8192 and L=1024, steps of at least 1024 tokens had a median interval of 16.5 ms, while a single 1024-token chunk took a median of at least 19.5 ms at every context we measured. Part 2's pre-registered check found that execution time tracked a step's tokens better than the interval did at C8 (Spearman 0.63 against 0.57), though still below the threshold. By execution time, large steps took 6.6–29.6× as long as small ones, with per-series Spearman correlations of 0.63–0.89 (post hoc). No C32 run recorded execution time. Taking each C32 step's cost from the next step's interval, which matches C8 execution time within 3% for large steps, gives 9.9× and a Spearman correlation of 0.92. The pre-registered verdicts stand.

Packing (post hoc). At B=2048 and L=1024, 94% of steps carrying at least 1024 prompt tokens held two or more prompts, and the median prompt-carrying step was a full 2048 tokens. Because the scheduler fills the budget with waiting prompts, a shorter prompt changes how often steps carry prompt work (38% of

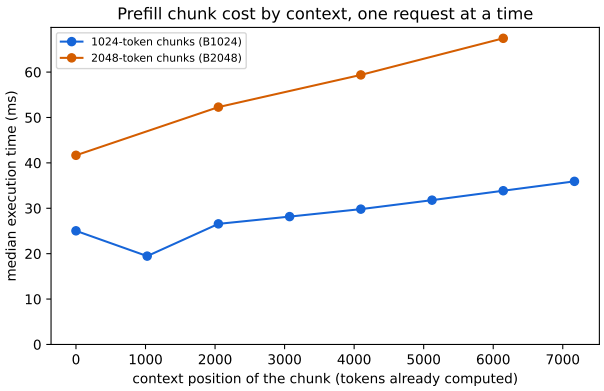


Figure 3: Median execution time of prefill chunks by context position, one request at a time with 8192-token prompts (300 chunks per point).

steps at L=1024, against 77% at L=2048) more than how large those steps are. The median therefore fell far (6.1 against 55 ms) while the tail fell only 19%. Our workload favors packing: all requests have the same lengths and start together, so they finish and return in groups.

Context. The pilot had suggested a role for context, because 2048-token steps were followed by shorter intervals at shorter prompts (45 ms at L=1024, 55.5 ms at L=2048). We withdraw that reading: at both lengths those steps start at context 0, and the gap follows the share of steps that carry prompt work (38% against 77%), as a lagged measure would. A second pre-registered design tested context directly. One request at a time (C1) sent 8192-token prompts at budgets 1024 and 2048, so every prompt step held one chunk of exactly B tokens at a known context position, timed by execution time (Figure 3). Chunk cost rose almost linearly with context. 1024-token chunks took 25.0 ms at context 0, 19.5 ms at context 1024, and 35.9 ms at context 7168; 2048-token chunks rose from 41.7 ms at context 0 to 67.4 ms at context 6144. The deepest chunk cost 1.44× and 1.62× the first (Spearman 0.96 in both series). From context 2048 on, each additional 1024 tokens of context added about 1.9 ms to a 1024-token chunk and 3.8 ms to a 2048-token chunk.

These slopes match a FLOP count. A chunk of n tokens at context c spends about $4ncd$ attention FLOPs per layer on the prior context, with d=896 and 24 layers, so each 1024 tokens of context costs 0.09 TFLOP for a 1024-token chunk and 0.18 TFLOP for a 2048-token chunk. That is 1.2 and 2.4 ms at the tensor peak, and 2.3 and 4.6 ms at the 40 TFLOPS that the context-0 2048-token chunk achieved. The measured slopes lie between the two and double with chunk size. The first chunk of each prompt was slower than the second at B=1024 (25.0 against 19.5 ms), and we do not know why; per-request setup does not explain it, because at C8 a lone 1024-token chunk at context 0 took 16.2 ms. The slow first chunk lowers ratios measured from context 0: taken from the second chunk, the B=1024 ratio is 1.84×, and a 2048-token chunk at context 0 cost 1.67× a 1024-token chunk, where FLOPs predict about 2.1×.

A step-cost model (post hoc). We fit execution time as a fixed cost plus terms for scheduled tokens, the attention pairs of prompt chunks, $\sum n(c + n/2)$, and the context of decoding requests. Over all 30,789 Part 2 steps the fit has $R^2 = 0.993$, with linear and attention work at 43 and 42 TFLOPS. Fit only on the one-request context series, the model predicts the ITL p99 of all six C8 series within 5%, weighting each step by the decoding requests it carries. It overpredicts the C8 median at long prompts, because one-request data cannot identify the decode term. Execution time itself, weighted the same way, reproduces the client's ITL p50 within 2% and p99 within 6%.

7 A secondary finding: GPU contention

We also ran a co-located synthetic GPU job alongside the prefill-heavy workload, with a sham condition and a recovery baseline. The job was busy for 25% or 50% of each 100 ms period, its kernels ran 14% and 28% of the time, and the two processes shared the GPU by time-slicing. At the 50% setting, the job cost 13–15%

Table 3: Scored predictions by phase. Phase B scores ten signatures for each of two workload regimes. Milestone 5's C8 predictions are counted in Part 2. Split or qualified verdicts (Phase D predictions 2 and 7) count as mixed.

Phase	Scored	Confirmed	Mixed	Not confirmed	Not testable
B: workload regimes	20	14	0	5	1
C: concurrency crossover	5	3	0	2	0
C: GPU contention	7	4	2	1	0
D: sweep and token budget	11	6	2	3	0
Milestone 5, C32 block	7	6	0	1	0
Milestone 5, Part 2	9	7	0	2	0
Total	59	40	4	14	1

in end-to-end latency and slowed requests even at C1, where it slowed prefill more than decode (+22.5% against +11.3%). We have no tested explanation for that difference; a post hoc check found the GPU clock 3–4% lower under contention at C1 (median 1680 against 1740 MHz), which may account for part of the slowdown. Mean `nvidia-smi` utilization moved only 1–6 points, against a 3.5-point spread between identical control runs; it was already about 90% at C1 without contention, which left little room to move, so it is a poor detector of this interference [7].

Of seven predictions, four held, two were mixed, and one failed. Utilization stayed within 5 points at C1 but moved 6.0 points at C8. Queue time at C8 rose only 6%, less than prefill (+17%), against the prediction, and the share of latency before the first token did not rise, the one outright failure. We observed this contrast with added load only under our closed-loop client.

8 The prediction ledger

Table 3 counts every scored prediction by phase: of 59, 40 were confirmed, 4 mixed, 14 not confirmed, and 1 could not be tested. We group the misses by cause; the reports discuss each one, including those that fit no group, such as the order of TTFT across budgets.

The hypothesis was wrong. Prefill did not dominate the prefill-heavy workload, time before the first token did not dominate past saturation (Section 4), and short prompts were packed instead of making steps smaller (Section 6).

The instrument could not see the effect. The iteration-token histogram could not show per-step chunk sizes, so Phase D's manipulation check failed as specified; that miss led to the step log, whose interval then turned out to lag by one step. `nvidia-smi` utilization is a busy fraction averaged over the driver's own sample period [7], so no polling rate could resolve 40 ms prefills.

The scheduler and the load generator shaped the result. Queue growth past saturation was sub-linear (not confirmed), and contention raised queue time less than prefill time (mixed). Both fit a scheduler that serves running requests first and turns waiting into longer steps, and a closed-loop client that caps how many requests can wait.

A test was too strict for its data. The rank statistic of step size and interval cleared its threshold at C32 but not at C8. Forward and reverse condition orders differed by 0.1–2.5% in latency and throughput, enough to fail a strict range-overlap test in Phase B. With three runs per order, two ranges fail to overlap by chance one time in ten for each metric, so across about ten metrics a few failures were expected.

Two central results, the per-step crossover and the context measurements, were tested directly only because an earlier prediction failed or prompted a post hoc reading.

9 Cost

Through October 28, 2026, the study's GPU rental cost $2.04 in total, about one hundredth of a cent per measured request. This covers all seven measurement sessions, their setup, and three machines that failed before collecting data. The cost stayed low because the study used one small model on one mid-range GPU and rented it only for data collection; analysis and writing ran on a laptop.

10 Threats to validity

- *One setup.* All results come from one 0.5B model on one GPU type with one vLLM version. A decode step at C1 took about 3.9 ms against a 2.2 ms floor for reading the weights, so host-side and launch work is a large share of each step. Larger models, other GPUs, and other serving systems could change the results qualitatively as well as in scale.
- *Workload.* Closed-loop load bounds queues. Runs of 100 requests start with a burst and end with a drain, which inflates mean TTFT at C32 (Section 4). Requests have identical lengths and start together, which may favor packing. Prompts are random tokens, and repetitions replay the same seed.
- *Few repetitions.* Results are medians of three repetitions, which agreed within about 1% for ITL and throughput in most cells; one C8 run had a p99 of 86 ms against 120 ms for the others. Bootstrap intervals over requests are narrower than these ranges and understate run-to-run spread. Pooled step statistics treat serially correlated steps as independent.
- *Timing and clocks.* Step timings are taken at vLLM's scheduler and include host-side work; we did not check them against GPU-side timing. GPU clocks were not locked: under load the GPU ran near its 140 W limit, at 1440 to 1860 MHz depending on condition and repetition.
- *Several machines.* Anchor series reproduced earlier results within about 1% on different machines, so machine-to-machine differences were small where we could check them.

11 Related work

Sarathi-Serve introduced chunked prefill with a per-step token budget, showed that long prefills stall co-scheduled decodes, and noted that each chunk must read the KV cache of the chunks before it [3]; we measure the budget's trade and the per-chunk cost of context directly in vLLM. Vidur predicts iteration time from profiled operator costs, modeling prefill attention by prompt length and decode attention by KV-cache reads [2]; our per-chunk measurements are the kind of data such a model needs for chunked prefill. DistServe runs prefill and decode on separate GPUs to avoid their interference [11]; we study it within one engine. Etalon argues that averaged metrics such as TPOT hide jitter [1], and our median-versus-tail results are an instance. We follow Hoefler and Belli's rules for performance measurement [4] in repeating runs, alternating run orders where the design allowed, and reporting every result, but report no confidence intervals. Schroeder et al. show how closed-loop load generators hide queueing [9], which matches our contention and saturation results.

12 Conclusion and future work

On this setup, from C16 to C32 at the default budget, added delay went into per-step time while steps filled toward the budget's cap; with the smallest budget it went into the queue. The budget decides how prompt work is cut into steps and so trades median against tail latency, and concurrency decides how many steps carry prompt work. A step's cost follows its tokens and their context closely enough that a four-term model fit on one-request data predicts the C8 tail, but that result is post hoc, and the pre-registered test of the token-count relation failed at C8. Four directions follow:

- a pre-registered test of the step-cost model in the style of Vidur [2]: predict ITL p50 and p99 at C32 and other budgets before measuring them, and identify the decode term with multi-request data;
- open-loop load with varied lengths, to see whether the per-step account and packing hold when queues can grow and requests do not arrive in groups;

- larger models and multi-GPU serving, where communication adds a third component to step cost;
- measurement-driven backend selection, which we deferred because single-server data cannot evaluate a selection policy without invented rewards.

References

[1] Amey Agrawal, Anmol Agarwal, Nitin Kedia, Jayashree Mohan, Souvik Kundu, Nipun Kwatra, Ramachandran Ramjee, and Alexey Tumanov. Etalon: Holistic performance evaluation framework for LLM inference systems, 2024. URL https://arxiv.org/abs/2407.07000.

[2] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S. Gulavani, Ramachandran Ramjee, and Alexey Tumanov. Vidur: A large-scale simulation framework for LLM inference. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 6, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/b74a8de47d2b3c928360e0a011f48351-Abstract-Conference.html.

[3] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134. USENIX Association, 2024. ISBN 978-1-939133-40-3. URL https://www.usenix.org/conference/osdi24/presentation/agrawal.

[4] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15)*, pages 73:1–73:12. ACM, 2015. doi: 10.1145/2807591.2807644.

[5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, pages 611–626. ACM, 2023. doi: 10.1145/3600006.3613165.

[6] NVIDIA. NVIDIA RTX A4000 datasheet. NVIDIA, . URL https://www.nvidia.com/content/dam/en-zz/Solutions/products/workstations/nvidia-rtx-a4000-datasheet.pdf. Accessed 2026-10-28.

[7] NVIDIA. nvidia-smi. NVIDIA documentation, . URL https://docs.nvidia.com/deploy/nvidia-smi/index.html. Accessed 2026-10-28.

[8] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 5, pages 606–624, 2023. URL https://proceedings.mlsys.org/paper_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html.

[9] Bianca Schroeder, Adam Wierman, and Mor Harchol-Balter. Open versus closed: A cautionary tale. In *3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 06)*, pages 239–252. USENIX Association, 2006. URL https://www.usenix.org/legacy/event/nsdi06/tech/schroeder.html.

[10] vLLM Project. Optimization and tuning: Chunked prefill. vLLM documentation. URL https://docs.vllm.ai/en/latest/configuration/optimization.html. Accessed 2026-10-28.

[11] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210. USENIX Association, 2024. ISBN 978-1-939133-40-3. URL https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin.